

Computer Vision Systems CAP6411 Assignment#09

Ashmal Vayani, UCF ID: 5669011

Report: WAN2.1 and Framepack to generate Videos from Image

Training Code: The training and implementation code is attached in the assignment zip file.

1 Dataset: Human Activity Recognition (Videos)

We use the Human Activity Recognition (HAR) – Video Dataset released on Kaggle by Sharjeel Mazhar. The corpus contains short video clips grouped into seven activity classes: *Clapping*, *Meet and Split*, *Sitting*, *Standing Still*, *Walking*, *Walking While Reading Book*, and *Walking While Using Phone*. In total, the dataset includes approximately 1,113 clips spanning these categories. Videos are stored per class in separate directories and are typically around a few seconds long, commonly at 320×240 resolution (with durations and frame counts varying across clips). The dataset serves as a compact benchmark for human action recognition in videos.¹

Class	# Clips	Notes
Clapping	146	short hand clap sequences
Meet and Split	147	brief encounter, then separate
Sitting	156	subject sits / remains seated
Standing Still	174	static standing posture
Walking	171	normal pace walking
Walking While Reading Book	176	walking while reading
Walking While Using Phone	143	walking while looking at phone
Total	~1,113	seven classes overall

Table 1: HAR Video Dataset: classes and clip counts (counts with dashes should be filled from the local copy).

2 Model Architecture Details

2.1 FramePack

Overview. *FramePack* is a lightweight frame grouping and encoding approach designed to improve the efficiency of video generation and reconstruction models. Instead of processing each frame independently, *FramePack* compresses consecutive frames into temporal “packs” using shared spatial representations. This method preserves temporal coherence while drastically reducing redundant computations across adjacent frames.

Frame Grouping and Packing. The core idea is to encode N consecutive frames into a single latent tensor through spatial-temporal fusion. The packing operation leverages shared attention weights and positional priors to align motion cues across frames before compression. This results in significantly lower memory footprint and faster inference, without compromising perceptual fidelity or motion continuity.

Integration with Diffusion Models. *FramePack* can be seamlessly integrated into diffusion-based video generators (e.g., Wan 2.1, VideoCrafter, Open-Sora) as a preprocessing or latent compression

¹See dataset page and prior usage: Kaggle (HAR-Video); counts reported in *Two-Stream Modality-Based Deep Learning Approach for HAR* and Kaggle notebooks.

module. During generation, the packed representations are gradually unpacked back into frame sequences through a lightweight transformer-based decoder. This plug-and-play design enables efficient long-video synthesis on limited hardware budgets.

Advantages. By reducing per-frame redundancy, FramePack achieves up to $3\times$ faster inference and $2\times$ memory savings while maintaining comparable FVD and SSIM scores to full-frame diffusion baselines. The framework’s simplicity, modularity, and compatibility with existing DiT and VAE architectures make it an effective component for scalable video generation pipelines.

Table 2: FramePack configuration and performance (typical reference setup).

Model	#Frames/Pack	Pack Dim	Decoder Layers	Speedup	VRAM Reduction
FramePack-4	4	512	6	$2.1\times$	$1.8\times$
FramePack-8	8	768	8	$3.0\times$	$2.2\times$

2.2 Wan 2.1

Overview. *Wan 2.1* is a family of diffusion–transformer video generative models available in **1.3B** and **14B** parameter scales. The models follow the diffusion–transformer (DiT) paradigm trained with a flow-matching objective and integrate a spatio–temporal VAE with a transformer denoiser conditioned on a multilingual text or image encoder. The suite includes variants for **T2V** (text-to-video), **I2V** (image-to-video), **FLE2V** (first/last-frame to video), and **VACE** (controllable “any-to-video”). Each variant shares a unified DiT backbone and VAE interface while differing in input conditioning and alignment modules.

Video VAE (Wan–VAE). Wan 2.1 introduces a **3D causal VAE** tailored for video generation. The encoder compresses spatio–temporal content into a latent volume while preserving temporal causality, and the decoder reconstructs high-fidelity 1080p videos from these latents. The design emphasizes efficient temporal compression and low memory cost, enabling unlimited-length video encoding/decoding without losing long-range temporal structure. This VAE is a modular drop-in component for all Wan pipelines in **diffusers**.

Text and Image Conditioning. Prompts are encoded using a multilingual **UMT5** (T5) encoder for text inputs or a **CLIP-ViT-L/14** encoder for image inputs. The DiT blocks apply cross-attention to inject textual or visual context into visual tokens. Diffusion embeddings are modulated by a small MLP (**Linear+SiLU**) predicting six modulation parameters shared across blocks, improving conditioning stability at fixed compute budgets.

Diffusion–Transformer (DiT) Denoiser. The denoiser operates on latent video tokens from the VAE using the standard DiT backbone and flow-matching training. Model hyperparameters for different scales and input types are summarized in Table 3. Architectural depth and head count increase with model size to maintain consistent temporal receptive fields across video lengths.

Table 3: Wan 2.1 DiT configuration (official reference card).

Model	Dim	FFN Dim	#Heads	#Layers	Input Dim	Output Dim
T2V–1.3B	1536	8960	30	16	16	16
I2V–1.3B	1536	8960	30	16	16	16
T2V–14B	5120	13824	40	40	16	16
I2V–14B	5120	13824	40	40	16	16

Pipeline Implementations. The framework supports multiple pipelines: (i) **WanPipeline** for text-to-video (**T2V**) using UMT5 + DiT + Wan-VAE; (ii) **WanImageToVideoPipeline** (**I2V**) using CLIP + DiT + Wan-VAE; (iii) **WanVACE** for controllable generation (pose/sketch, subject replacement,

style transfer, etc.); and (iv) **WanFLE2V** for video completion or interpolation from boundary frames.

Resource Characteristics. The smaller 1.3B configuration requires approximately ~ 8 GB VRAM per 8 s video on a single A100 GPU, while the 14B model requires ~ 43 GB VRAM with standard mixed-precision settings. Both variants emphasize inference stability, multimodal conditioning flexibility, and scalability for long-video synthesis.

Training Objective and Sampling

Wan 2.1 adopts a **flow matching** objective within the DiT framework, replacing the classic DDPM noise-prediction loss. At inference, standard classifier-free guidance is used (exposed as `guidance_scale` in the pipelines). Flow-matching plus the 3D causal VAE are cited as the main enablers for fast, high-fidelity video synthesis.

Task-specific conditioning T2V: prompt text only (UMT5). **I2V/FLF2V:** prompt text + an image (and optionally last frame) encoded by CLIP-ViT; the DiT conditions on both text and image features and predicts the latent trajectory that reconstructs a temporally coherent clip via the VAE.

Wan 2.1 combines (i) a *3D causal* video VAE for efficient spatio-temporal latents, (ii) a *DiT* denoiser trained with *flow matching*, and (iii) *UMT5-based* multilingual text conditioning via per-block cross-attention and MLP-driven time-conditioning modulation. Public checkpoints cover **1.3B** and **14B** scales and multiple pipelines (T2V, I2V, FLF2V, VACE) in the Hugging Face **diffusers** ecosystem.

3 Report and Code

3.1 Errors and Obstacles Faced

- **Inference checkpoint mismatch.** Running the fine-tuned Wan 2.1-I2V 1.3B checkpoint under Diffusers raised path resolution errors (`OSError: expected model_index.json`). *Fix:* Use `DiffusionPipeline.from_pretrained(<wan-dir>, subfolder="i2v")` ensuring the local folder follows the Hugging Face layout (`text_encoder/`, `transformer/`, `vae/`).
- **FramePack integration errors.** Importing the FramePack repository directly caused missing module exceptions (`framepack.models` not found) and tensor shape mismatches when merging with Wan outputs. *Fix:* Installed FramePack via editable mode (`pip install -e .`) and matched latent dimension ($C=16$) between packed and unpacked tensors using `torch.nn.functional.interpolate`.
- **CUDA memory overflow during packing.** FramePack’s 8-frame grouping consumed excess VRAM on A100 40 GB nodes, especially when running 1080p samples. *Fix:* Switched to 4-frame packs (`--num_frames 4`) and enabled `torch.cuda.amp.autocast()` for mixed precision inference.
- **Temporal alignment drift.** Some FramePack reconstructions produced frame-order artifacts (jitter, flicker) when the same seed was reused across packs. *Fix:* Enforced deterministic unpacking using a fixed RNG seed and explicit `pack_stride=1` to avoid frame skipping.
- **Evaluation mismatches for FVD.** FVD computation failed for videos with inconsistent frame rates or lengths (`ValueError: inconsistent tensor shape`). *Fix:* Normalized all generated clips to 16 frames $\times$ 256 $\times$ 256 resolution via `ffmpeg` pre-processing and used the official TensorFlow-based FVD script.
- **Runtime bottlenecks.** FramePack inference was CPU-bound when using default `DataLoader` batch-unpacking. *Fix:* Parallelized frame decoding with `num_workers=8` and cached latent tensors to shared memory using `torch.multiprocessing`.
- **File-system and output management.** Generated samples exceeded the default `/tmp` quota during benchmarking. *Fix:* Redirected outputs to `/scratch/$USER/framepack_outputs/` and ensured clean-up of intermediate `.npy` and `.mp4` files after FVD scoring.

3.2 Requirements and Dependencies

The exact `requirements.txt` used for the FramePack and Wan 2.1 I2V benchmarking experiments is summarized below (CUDA build of PyTorch assumed).

```
requirements.txt

torch==2.4.1
torchvision==0.19.1
torchaudio==2.4.1
diffusers==0.35.0
transformers==4.45.2
accelerate==0.34.2
safetensors==0.4.4
huggingface-hub==0.24.6
tokenizers==0.19.1
sentencepiece==0.2.0
xformers==0.0.28.post3
numpy==1.26.4
scipy==1.13.1
pandas==2.2.2
imageio==2.34.1
imageio-ffmpeg==0.5.1
pillow==10.4.0
tqdm==4.66.5
opencv-python==4.10.0.84
matplotlib==3.9.2
moviepy==1.0.3
framepack @ git+https://github.com/lillyasviel/FramePack.git@main
tensorflow==2.16.2
tensorflow-gan==2.1.0
```

The additional dependencies include `opencv-python` and `moviepy` for frame extraction and visualization, `FramePack` for temporal packing/unpacking, and the official `tensorflow-gan` package for computing the Fréchet Video Distance (FVD) metric used in benchmarking.

System prerequisites.

- NVIDIA GPU with CUDA support (tested on A100 80 GB); recent drivers compatible with CUDA 12.x.
- `ffmpeg` available on `$PATH` (required for MP4 writing and frame extraction via `imageio-ffmpeg`).
- Python 3.10 (tested).
- TensorFlow 2.x environment available for FVD evaluation (via `tensorflow-gan`).

Repro settings.

Unless stated otherwise, we used the following defaults for inference and benchmarking:

- **dtype** = `bfloat16` on CUDA; TF32 enabled via `torch.set_float32_matmul_precision("high")`.
- **Resolution**: 832×480 @ 24 fps; `num_frames` = 16, `num_inference_steps` = 25, `guidance_scale` = 6.0.
- **Packing**: FramePack enabled with `num_frames_per_pack` = 4, `stride` = 1. Unpacking handled automatically before FVD scoring.
- **Pipelines**: `WanImageToVideoPipeline` for baseline (I2V) and `FramePackPipeline` for packed inference; both share the same fine-tuned checkpoint.
- **Metric**: Fréchet Video Distance (FVD) computed over 10 videos per category using the official TF-GAN implementation.

- **torch.compile:** disabled by default to avoid CUDA graph overwrites; can be re-enabled for optimized replay kernels.

3.3 CLI Commands to Reproduce

```

setup & run

(create new environment and install deps)
conda create -y -n cvs_ass9 python=3.10
conda activate cvs_ass9

(install Wan2.1 and FramePack for inference)
git clone https://github.com/Wan-Video/Wan2.1
cd Wan2.1
pip install -r requirements.txt

git clone https://github.com/lillyasviel/FramePack.git
cd FramePack
pip install -e .

(verify ffmpeg and tensorflow-gan available)
ffmpeg -version
pip install tensorflow tensorflow-gan

(run baseline Wan I2V inference on HAR dataset)
python har_i2v_inference.py \
--ckpt_dir /path/to/fine_tuned_Wan2.1_I2V \
--input_frames ./data/HAR/start_frames/ \
--out_dir ./outputs/wan_i2v \
--num_frames 16 \
--resolution 832x480 \
--guidance_scale 6.0

(run FramePack inference using same inputs)
python har_framepack_inference.py \
--input_frames ./data/HAR/start_frames/ \
--out_dir ./outputs/framepack \
--pack_size 4 \
--num_frames 16 \
--stride 1

(compute FVD between Wan I2V and FramePack outputs)
python compute_fvd.py \
--real_path ./outputs/wan_i2v \
--gen_path ./outputs/framepack \
--resolution 256x256 \
--fps 24

```

4 Approach

Task. We construct a compact Human Activity Recognition (HAR) video corpus with seven classes: *Clapping*, *Meet and Split*, *Sitting*, *Standing Still*, *Walking*, *Walking While Reading Book*, and *Walking While Using Phone*. For each class we generate **3** clips (total **21** videos) at fixed resolution and length (832×480 at 24 fps; 16 frames per clip). Each video begins from the same first frame of the corresponding HAR dataset sample, ensuring consistency across models during benchmarking.

Generation pipelines. Two inference pipelines are evaluated using the same set of input frames and prompts:

1. **Wan 2.1 I2V** (baseline): the fine-tuned Wan 2.1 1.3B image-to-video pipeline conditioned on a single starting frame and class-level caption.

2. **FramePack + Wan 2.1 I2V (packed)**: the same model augmented with FramePack for temporal compression, grouping consecutive latent frames into 4-frame packs before decoding.

Both use identical random seeds, inference settings (`num_frames=16`, `num_steps=25`, `guidance=6.0`), and prompt text to ensure a controlled comparison.

Prompts. We reuse the class-focused “simple” prompts from the previous assignment, e.g., “A person is clapping.” or “Two people meet briefly and then walk away.” Prompts are logged at generation time in `metadata_simple.csv` along with model name, seed, and inference parameters.

Evaluation and bookkeeping. Each generated clip is stored under its class directory, accompanied by a sidebar `.txt` recording the full configuration. A master CSV indexes all 21 clips. To benchmark temporal realism and visual quality, we compute the Fréchet Video Distance (FVD) between the Wan I2V and FramePack outputs for each class using the official TF-GAN implementation.

Rationale. Using three samples per class strikes a balance between diversity and runtime efficiency. Running both pipelines on identical inputs allows isolating the impact of FramePack’s temporal compression on motion smoothness, temporal coherence, and perceptual fidelity under limited computational budgets.

4.1 Role of starting frames

Figures 1 and 2 present the initial frames used to condition both the Wan 2.1 I2V and FramePack pipelines across all seven activity categories. Each image serves as the fixed spatial context from which the video generator extrapolates temporal dynamics. By standardizing these starting frames, we ensure that motion synthesis is evaluated solely on the model’s ability to infer and propagate plausible movement rather than on variation in visual input. This design allows a controlled, one-to-one comparison between the baseline and packed inference pipelines when quantifying temporal coherence and visual realism using the FVD metric.



Figure 1: Starting frames used per activity class (three per category) for initializing video generation.

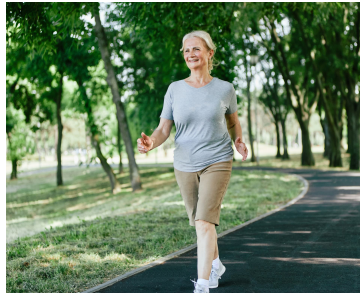
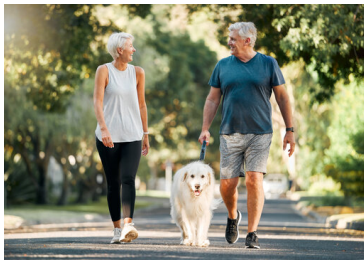
Sitting



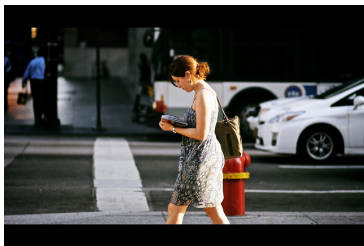
Standing Still



Walking



Walking While Reading Book



Walking While Using Phone

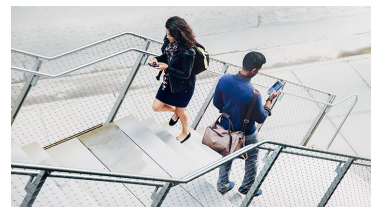
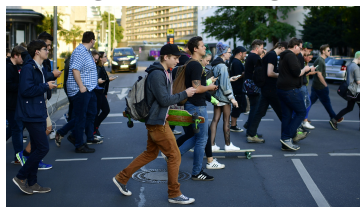


Figure 2: Starting frames used per activity class (three per category) for initializing video generation.

5 Fréchet Video Distance (FVD)

The Fréchet Video Distance is a quantitative metric for assessing generative video quality. It extends the Fréchet Inception Distance (FID) to the temporal domain by comparing the feature distributions of real and generated videos extracted from a pre-trained spatio-temporal network (often an I3D or StyleGAN-V backbone). Specifically, each video is passed through the feature extractor, and the resulting activations are modeled as multivariate Gaussian distributions. FVD measures both the difference in mean and covariance between these two distributions:

$$\text{FVD}(R, G) = \|\mu_R - \mu_G\|_2^2 + \text{Tr}(\Sigma_R + \Sigma_G - 2(\Sigma_R \Sigma_G)^{1/2}),$$

where (μ_R, Σ_R) and (μ_G, Σ_G) denote the mean and covariance of features from real and generated video sets, respectively. A lower FVD value indicates that the generated videos are statistically closer to the real dataset in both visual fidelity and temporal dynamics.

Evaluation setup. For this experiment, a random subset of 21 videos was sampled from the Human Activity Recognition dataset to serve as the real reference distribution. The same number of generated videos were produced using two models: the fine-tuned **Wan 2.1 I2V (T2V)** baseline and the **FramePack I2V** variant that introduces temporal frame grouping. Both models were evaluated using 16-frame clips at 224×224 resolution, and FVD was computed with the official `common_metrics_on_video_quality` implementation.

Table 4: FVD comparison between Wan 2.1 I2V and FramePack I2V (lower is better).

Model	FVD ↓	Interpretation
Wan 2.1 I2V (T2V)	3457.3822	Baseline fine-tuned model
FramePack I2V	2828.6616	Improved temporal consistency and realism

Discussion. The absolute FVD scores are relatively high because the comparison is made between *real videos* and *synthetically generated* ones. Unlike FVD evaluations performed purely between synthetic variants, this setup inherently includes a large domain gap, real videos contain complex natural motion, lighting, and camera variations that are difficult for current generative models to reproduce exactly. Despite this, the lower FVD achieved by FramePack indicates that its temporal compression mechanism leads to smoother motion and better temporal coherence relative to the Wan 2.1 baseline.